

DocMind 企业文档处理智能体 — 技术方案文档

版本: v2 (Agent 架构) · 分支: feat/agent-v2 (默认) · 面向对象: 团队同事 / 评审
更新至最新: 6 组 Tool · 检索改 SQL 直查 · 报表生成 · 深色聊天界面

1. 项目目标

构建一个 **企业单据处理智能体 (Chatbot)**，覆盖"识别 → 人工校验 → 入库 → 统计问答 → 可视化 → 报告"完整链路：

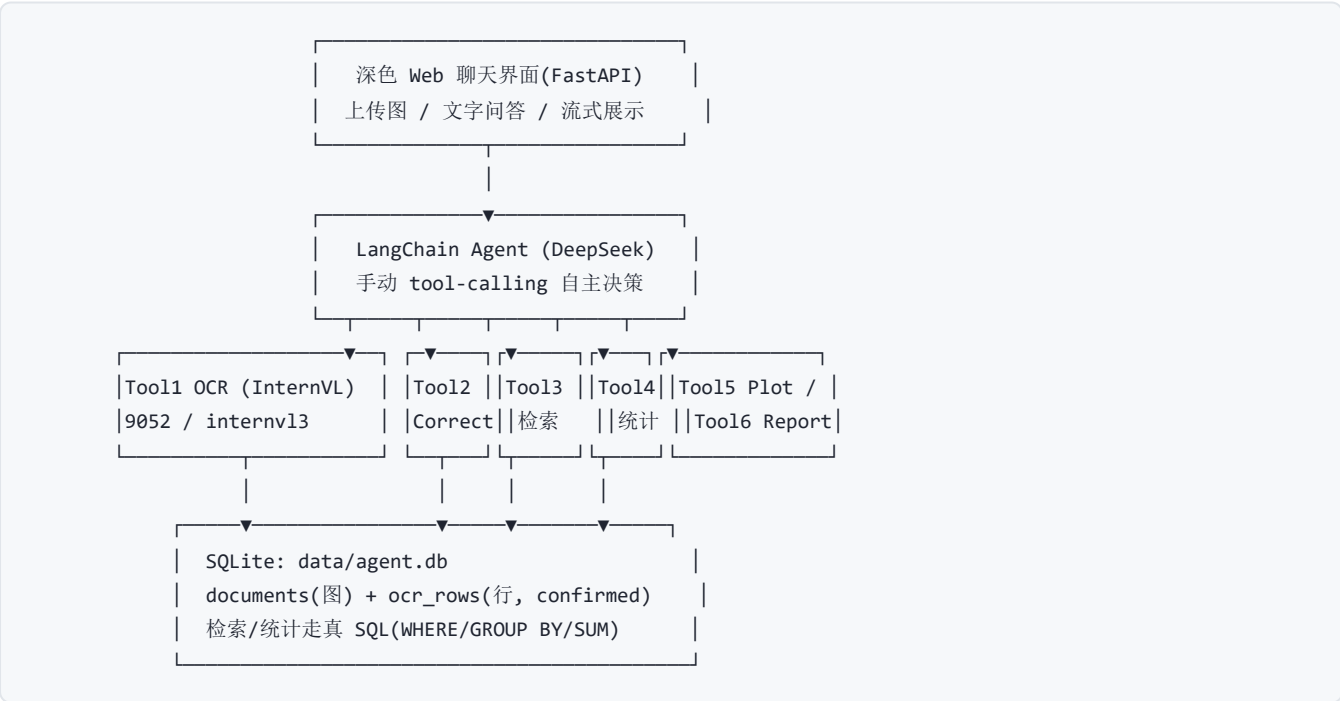
- 用户上传**支票/销售单据图片**，系统自动结构化识别
- 识别结果**人工可改**（模型准确率 ~80%，Agent 需要 100% 准确数据）
- 多张图数据入库，供**精确检索与统计**（SQL 直查，避免全量喂大模型烧 token）
- 统计结果可**画图**、可**生成 Word 报告**
- 能力**可插拔扩展**（加一个 @tool 即扩展）

1.1 目标用户链路

上传图片 → OCR识别 → 人工核对/修改 → 确认入库 → 提问统计 → 图表 → Word报告

2. 总体架构

核心思想：以 Agent 为中心（而非多个独立服务）。 用 LangChain 搭建 DeepSeek LLM 智能体，为其"加装"一组 Tool，每个 Tool 补齐 DeepSeek / 基础模型的一项短板。



2.1 为什么 6 个 Tool (每个补一个短板)

Tool	短板	解决方案	关键技术
OCR	DeepSeek 是"盲"的, 无法看图	InternVL 微调模型识别图片	InternVL3 @ 9052 (OpenAI 兼容)
Correct	OCR 准确率仅 ~80%	界面/Agent 人工核对修改识别结果	SQLite 行级状态机
检索	多图全喂费 token 且不准	**SQL 直查**明细	`WHERE item/年份 + LIMIT`
统计	数字要汇总	**SQL 聚合**	`GROUP BY + SUM`
Plot	统计不直观	绘制统计图	matplotlib → PNG
Report	统计要交付	生成 Word 报告	python-docx → docx

关键：销售数据是**结构化表格**，非文档语义 —— 检索统计直接用 SQL，**不需要向量检索**。

3. 模块设计

3.1 目录结构

```
enterprise_ocr_service/
├─ config.py           # .env 加载(OCR 9052 / DeepSeek key)
├─ agent/
│   ├─ llm.py          # ChatDeepSeek 封装
│   └─ agent.py        # 手动 tool-calling 循环 + 6 工具 + 流式事件
├─ tools/              # 可插拔工具
│   ├─ ocr_client.py   # InternVL 客户端(cols8 中英语义 prompt)
│   ├─ ocr_tool.py     # Tool1: 识别 → 入库待确认
│   ├─ correct_tool.py # Tool2: list/show/update/confirm
│   ├─ rag_tool.py     # Tool3/4: 检索+聚合(真 SQL, 支持 year)
│   ├─ plot_tool.py    # Tool5: 柱状/饼图 → charts/*.png
│   └─ report_tool.py  # Tool6: Word 报告 → reports/*.docx
├─ db/
│   ├─ database.py     # SQLite(标准库 sqlite3, WAL)
│   ├─ schema.py       # documents + ocr_rows 建表
│   └─ crud.py         # CRUD + SQL 直查函数
├─ web/
│   ├─ agent_chat.py   # SSE 流式端点 /api/agent/chat(支持图片)
│   ├─ agentchat.html  # 深色工业风聊天界面
│   └─ main.py         # 入口 + 其它 /api/* 端点
├─ run_web.py / start_web.py # 启动
├─ requirements.txt    # 依赖清单
├─ data/ charts/ reports/ # 数据/图表/报表(gitignore)
└─ tests/test_agent_v2.py # 单元测试
```

3.2 数据模型 (SQLite)

documents (每张上传图片) : id, file_name(唯一), sha256, uploaded_at, status

ocr_rows (每张图识别行) : id, doc_id(FK), seq, desc/date/from/item/amount/price/tax/sum, status(pending/confirmed), modified_by, modified_at

只有 **confirmed** 行进入统计。date 存 YYYY年MM月DD日，年份过滤用 substr(date,1,4)。

3.3 检索/统计: SQL 直查 (核心设计)

```
-- 明细: 关键词 + 年份过滤
SELECT * FROM ocr_rows r JOIN documents d ON d.id=r.doc_id
WHERE r.status='confirmed'
      AND (r.item LIKE ? OR r.desc_ LIKE ? OR r.from_ LIKE ? OR d.file_name LIKE ?)
      AND substr(r.date_,1,4)=?          -- 年份(可选)
ORDER BY r.date_ DESC LIMIT ?;

-- 聚合: GROUP BY + SUM
SELECT item, COUNT(*), SUM(CAST(amount AS REAL)), SUM(CAST(sum_ AS REAL))
FROM ocr_rows WHERE status='confirmed' [AND ...]
GROUP BY item ORDER BY total DESC;
```

- 只把聚合后的几行喂给 LLM —— 精确、省 token、速度快
- crud 提供 `search_confirmed` / `summarize_confirmed` / `confirmed_count` (带 keyword/year 过滤)

3.4 Agent 中枢

- LLM: DeepSeek `deepseek-chat` (langchain-deepseek)
- 编排: 手动 tool-calling 循环 (`run_agent_events`), 产出流式事件

`intent` → `tool_call` → `tool_result` → `answer`, 供聊天界面实时展示

- 系统提示 (`EVENTS_SYSTEM_PROMPT`) 明确各工具何时用; 问题含年份→传 `year`
- 扩展: 加一个 `@tool` 并放入 `_TOOLS` 列表即可

4. 关键技术结论 (实测沉淀)

1. InternVL prompt 需中英文语义引导: 纯英文 key 会把 `from` 认成日期;

`from`(发货公司/源公司) 中英混写识别正确 (S1 真机实测)

1. 识别数据必须人工确认后才进统计: 模型有幻觉/重复行, 草稿不入库统计
2. 单槽 llama.cpp: 一张图 20~60s, 一次一个请求, 长 prompt 会卡服务
3. 检索用 SQL 不用向量: 结构化表格 GROUP BY/SUM 直接精确聚合, 省 token
4. langchain 1.x: `AgentExecutor` 在 `langchain_classic.agents`; 流式手写 tool-calling 循环
5. python-docx / matplotlib 中文需中文字体, docx 默认 Arial 兼容好
6. PyPI 直连不通 → 清华源 `-i https://pypi.tuna.tsinghua.edu.cn/simple`

5. 运行与联调

```
# 依赖
pip install -i https://pypi.tuna.tsinghua.edu.cn/simple -r requirements.txt

# .env: OCR_BASE_URL=http://127.0.0.1:9052/v1 OCR_MODEL=internvl3 DEEPSEEK_API_KEY=sk-xxx
```

```
# 启动(需 9052 InternVL 在线)
python run_web.py          # http://127.0.0.1:8100
```

5.1 对话示例 (Agent 自主调工具)

你说	Agent 做什么
上传一张销售图	调 `ocr_recognize` → 识别入库(待确认)
"确认"	调 `correct_confirm` → 进入统计
"2022年销售额如何"	调 `rag_summarize(year='2022')` → SQL 精确返回
"画个柱状图"	调 `plot_chart` → 图直接显示在对话
"生成一份2022年统计报告"	调 `generate_report(year='2022')` → Word 报告

5.2 API 一览

方法 & 路径	用途
POST `/api/agent/chat`	SSE 流式对话 (支持图片上传)
POST `/api/ocr`	上传图片识别 → 入库待确认
GET `/api/docs` · `/api/docs/{name}`	单据列表 / 某图识别行
POST `/api/rows/{id}`	人工修改某行
POST `/api/confirm`	确认某图全部行
POST `/api/chart`	画统计图
GET `/charts/{file}`	图表文件
GET `/api/stats`	总览

6. 测试与质量

```
pytest tests -q          # db CRUD + 工具逻辑(离线)
ruff check db agent tools web config.py
```

7. 当前进度

项	状态
SQLite 数据层 (doc+rows/状态机)	✓
Tool1 OCR (InternVL 9052)	✓ 真实联调通过
Tool2 Correct (人工修改/确认)	✓
Tool3/4 检索统计 (SQL 直查, year 支持)	✓
Tool5 Plot (画图)	✓
Tool6 Report (Word 报告)	✓
Agent 自主决策 (6 工具, 流式事件)	✓
深色聊天界面 + 图片直入 + 图表内嵌	✓
发票等第二类目	📅 规划 (数据/模型未提供)

8. 版本演进 (git)

master

旧版：双 FastAPI 服务(8000 OCR + 8100 rag/CSV) ← 归档

feat/agent-v2

新版：Agent 架构(本文档) ← 默认分支

注：曾尝试接入同事的 DocMind 前端契约(/v1 适配层 + task 模型)，因"只需借鉴风格、不绑定其契约"被回退；现仅保留深色工业风样式，逻辑为自有 Agent（详见 git tag docmind-experiment）。